# ChatGPT

GitHub Project Next Steps

You

im doing a sc2002 grp project and my teammate we decide to use github, from the branch we did at stage 2 **Xinyi/stage 2 features** , can you advice me on what to do next

Worked for 1m 3s

ChatGPT

I've looked through your GitHub repository, the merged Stage 2 pull request, and your SC2002 Restaurant Rush assignment brief.

My recommendation is: **do not start coding new features immediately. First, make sure your team has a stable Stage 2 version on `main`, then divide the remaining work into manageable branches.**

Your group is actually at a good point to move forward because Xinyi's Stage 2 pull request has already been merged.

## 1. Where your project currently stands

**Repository** Accessible

Chuanhhua/sc2002_group_assignment

Default branch: `main`

**Stage 2 — Xinyi's contribution**

PR #12 Merged

**Xinyi/stage 2 features**

Merged 12 November 2026

12 files changed · 671 additions · 327 deletions

**Take your curiosity further**
Scan to get the ChatGPT app

Merge commit: `7640683`

View Stage 2 pull request ↗

### Features introduced

- `VIPCustomer.java`
- `CriticCustomer.java`
- Customer-specific satisfaction loss
- VIP 10% discount
- Critic orders the most expensive dish
- Money stored in integer cents
- Seeded randomness for reproducible runs
- Improved console interface
- Stage 2 victory and defeat test inputs

These changes follow the Stage 2 requirements in your assignment brief.

However, I noticed something important.

**Your Stage 2 feature branch is now behind** `main` **.** It is 29 commits behind the current main branch, so you should not continue developing on that old branch.

There are also several existing development branches:

| Branch | What I found |
|---|---|
| `main` | Current shared development branch |
| `xinyi/stage-2-features` | Historical Stage 2 branch |
| `zongnan/stage-3` | Earlier Stage 3 work |
| `stage3_combomeal` | Earlier ComboMeal work |
| `xinyi/stage-3-fixes` | Stage 3 fixes |
| `caleb/stage2` | Another Stage 2 branch |

**Take your curiosity further**
Scan to get the ChatGPT app

The older Stage 3 branches are also behind `main`. I would avoid merging them directly into `main` without first checking and reconciling their changes.

## 2. What should you do next?

Your assignment has three separate stages.

According to the project briefing:

| Stage | Requirements | Git tag |
|---|---|---|
| Stage 1 | Basic restaurant operation | `stage-1` |
| Stage 2 | VIP and Critic customers | `stage-2` |
| Stage 3 | Host, ComboMeal and Happy Hour | `stage-3` |

You need three independently runnable versions, not just one final version.

I suggest following this sequence:

1. **Step 1 — Stabilise Stage 2**
   Make sure `main` contains the merged Stage 2 code and that the game compiles and runs correctly.
2. **Step 2 — Create Stage 2 checkpoint**
   Tag the verified Stage 2 version as `stage-2`, preserving it even after Stage 3 development starts.
3. **Step 3 — Start Stage 3 development**
   Create new feature branches from the updated `main`, not the old Stage 2 branch.
4. **Step 4 — Integrate Stage 3**
   Review each teammate's pull request, resolve conflicts and test the combined game.
5. **Step 5 — Finalise submission**
   Verify all three stage versions, test runs, UML diagrams, README, report and video.

**Take your curiosity further**
Scan to get the ChatGPT app

## 3. How I recommend dividing the work among five people

Since you have five group members, I recommend splitting Stage 3 into separate features instead of having everyone modify `GameController.java` simultaneously.

The Stage 3 requirements are:

### A. Host

- Seats at most one queued customer per turn.
- Works in addition to the Waiter's seating task.
- Host acts before the Waiter, allowing the newly seated customer to order in the same turn.

### B. ComboMeal

- Consists of two existing menu items.
- Preparation units are added together.
- Price is 90% of the combined price, rounded to the nearest cent.
- Customers choose combos according to a documented rule.

### C. Happy Hour

- Can be activated once per game.
- Lasts for the current turn and the next turn.
- Orders taken during this period receive 20% off.
- Waiting and unserved seated customers recover 15 satisfaction points at the start of each active turn.
- VIP discount is applied before the Happy Hour discount.

Here is my suggested division:

### Suggested five-person workload

#### Member 1 — Integration and main controller

Coordination

- Coordinate Stage 3 integration.
- Review `GameController.java` .
- Handle turn order and interactions between features.

Take your curiosity further
Scan to get the ChatGPT app

- Run integration tests.

Main files: `GameController.java`, `README.md`

### Member 2 — Host

- Implement `Host.java`.
- Implement seating logic.
- Integrate Host's action before the Waiter.

Main files: `Host.java`, related controller changes

### Member 3 — ComboMeal

- Implement `ComboMeal.java`.
- Update `Menu.java` to register combos.
- Implement the customer's combo-selection rule.

Main files: `ComboMeal.java`, `Menu.java`, related customer changes

### Member 4 — Happy Hour

- Implement the Happy Hour pricing policy.
- Implement satisfaction recovery.
- Handle activation and duration.

Main files: `HappyHour.java`, related controller changes

### Member 5 — Testing, documentation and UML

- Prepare Stage 3 victory and defeat test runs.
- Check Stage 1 and Stage 2 regression tests.
- Update UML class diagrams.
- Prepare the sequence diagram and report evidence.

Main files: `tests/`, UML diagrams, documentation

**Take your curiosity further**
Scan to get the ChatGPT app

One important consideration: members 2, 3 and 4 may need changes in shared classes such as `GameController.java` and `ConsoleUI.java`.

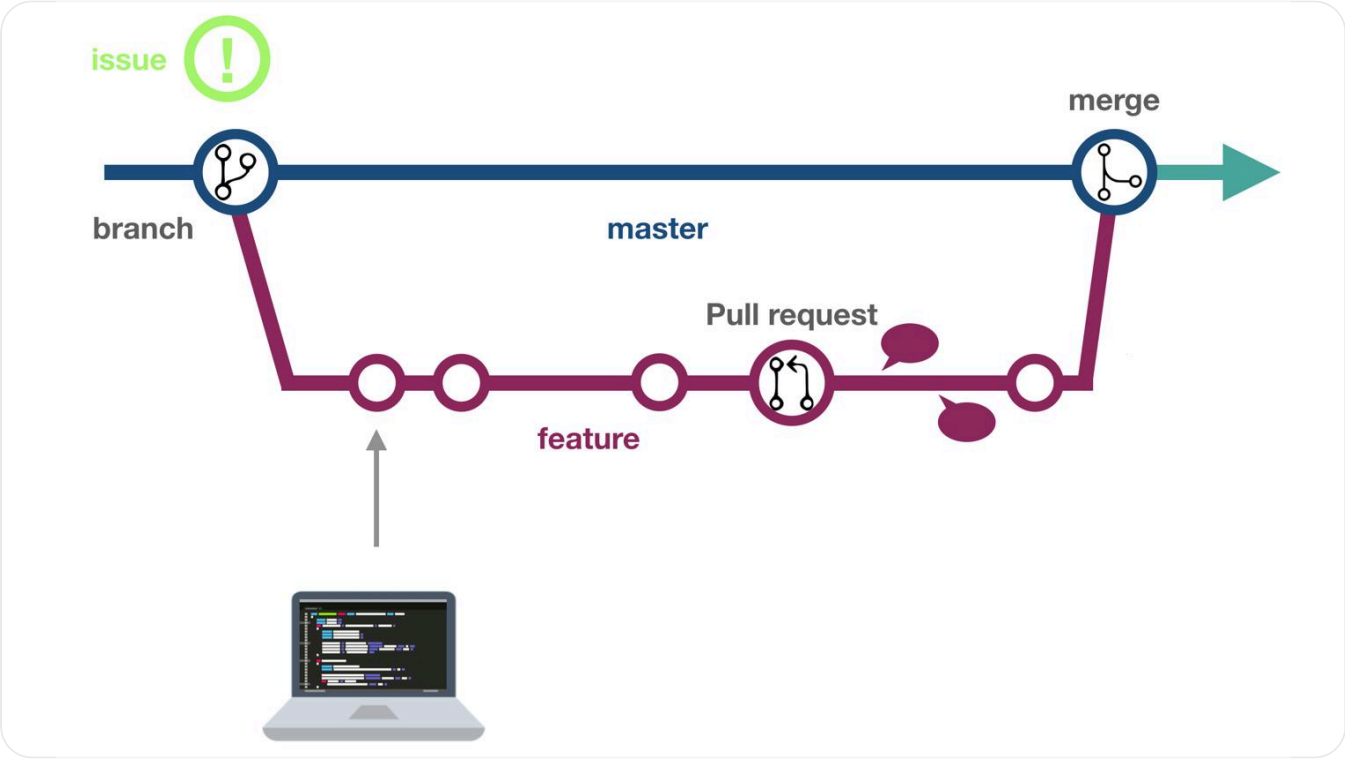
To avoid conflicts, agree on the methods and interfaces before coding. Each member should primarily own their feature class, while Member 1 coordinates changes to shared classes.

Also, Member 5 should not wait until everyone finishes coding. UML and testing should progress alongside implementation.

## 4. How to use GitHub properly from now onwards

This is probably the most important part for your group.

I recommend adopting this workflow:




**The basic idea:**

- `main` is your group's shared stable branch.
- Every member creates their own feature branch.
- Everyone commits their changes to their own branch.
- Members open pull requests to merge into `main`.

**Take your curiosity further**
Scan to get the ChatGPT app

- Someone reviews and tests the changes before merging.

For example:

| Member | Branch |
| --- | --- |
| Integration | `bryan/stage3-integration` |
| Host | `caleb/host-feature` |
| ComboMeal | `zongnan/combomeal-feature` |
| Happy Hour | `chuanhua/happy-hour` |
| Testing/UML | `xinyi/stage3-testing-uml` |

These are suggested names, not existing branches.

**Commands you should learn**

First, update your local `main`:

```
git checkout main
git pull origin main
```

Create your own feature branch:

```
git checkout -b bryan/stage3-integration
```

After making changes:

```
git add .
git commit -m "Integrate Stage 3 features"
git push -u origin bryan/stage3-integration
```

Then go to GitHub and open a Pull Request:

- Base: `main`

**Take your curiosity further**
Scan to get the ChatGPT app

- Compare: `bryan/stage3-integration`

Once reviewed and tested, merge it.

Before starting another task, update your local copy of `main` again.

**Important:** Don't let five people push directly into `main`. That makes it much harder to track mistakes and resolve conflicts.

## 5. Things I noticed that your group should address

There are a few items I would prioritise before submission.

| Item | Current observation | What to do |
|---|---|---|
| Stage 2 tag | README says `stage-2`, but I haven't independently verified the tag exists | Check GitHub Tags |
| Stage 1 tag | README references `stage-1` | Verify it remains runnable |
| Stage 3 tag | README references `stage-3` | Create after final testing |
| README | Already has a useful stage-by-stage structure | Verify all claims against actual code |
| Test cases | Stage 2 test inputs are listed | Verify they actually produce the documented results |
| AI history | README has placeholders for each member's branch | Each member needs to upload their AI conversations |

The assignment specifically requires three class diagrams (one per stage) and one sequence diagram showing a complex use case from Stage 2 or Stage 3.

It also requires evidence of what was added, what was modified, what still works and the effect on the design.

These are not optional extras. They are part of demonstrating your development process.

Sources: Your Restaurant Rush assignment brief, particularly pages 5–6 ↗, and the uploaded project briefing, pages 3–7.

**Take your curiosity further**
Scan to get the ChatGPT app

## 6. What I suggest you do today

I would work through these tasks in order:

**Your GitHub action checklist** 0/8

- [ ] Pull the latest main branch onto my computer.
- [ ] Compile and run the Stage 2 game.
- [ ] Verify the VIP and Critic behaviours.
- [ ] Verify the Stage 2 victory and defeat test inputs.
- [ ] Check that the stage-2 Git tag exists.
- [ ] Discuss and assign the five Stage 3 responsibilities.
- [ ] Create fresh Stage 3 feature branches.
- [ ] Agree on shared GameController and ConsoleUI interfaces.

## 7. Let me help you with the next step

Before we start changing code, I want to make sure we organise your group's GitHub properly.

**My recommendation is to start with your own role as the integration coordinator, rather than immediately taking ownership of a new feature.** This lets you understand the existing Stage 2 implementation and coordinate everyone's work.

One thing I want to establish first:

**Have all five of you already agreed on who is responsible for Host, ComboMeal, Happy Hour, integration and testing, or have you not divided the workload yet?**

Once we establish that, we can set up the exact branch structure and workflow, step by step, so nobody accidentally overwrites another person's work.

**Take your curiosity further**
Scan to get the ChatGPT app

You

can you teach me how to create a branch

ChatGPT

Of course! I'll teach you **step by step how to create your own branch on GitHub** for your SC2002 Restaurant Rush project.

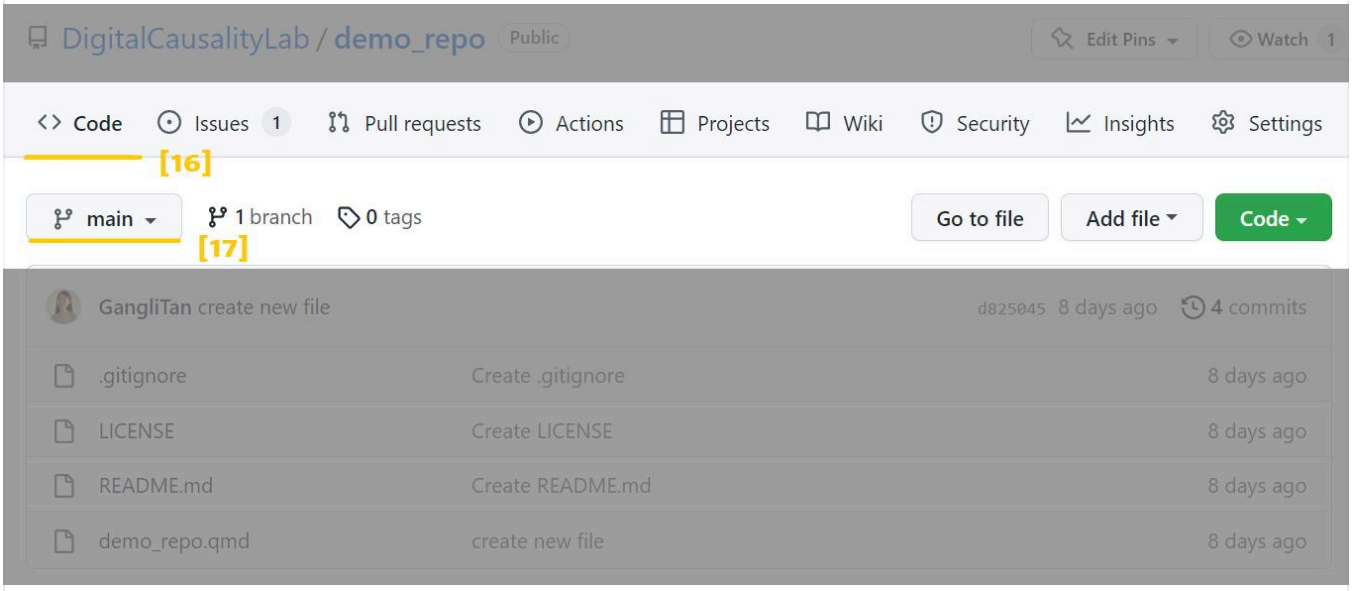
Since your group is collaborating, you should create your branch from the latest `main` so you have everyone's updated code.

## Step 1: Open your group's GitHub repository

Go to:

SC2002 Restaurant Rush GitHub Repository ↗

You should see something similar to this:




Look for the branch selector near the top-left of the file list. It should say `main`.

**Take your curiosity further**
Scan to get the ChatGPT app

### Step 2: Click the branch dropdown

1. Click the button labelled `main`.
2. A menu will appear with a search box.
3. Type your new branch name.

For example, if you're Bryan and working on Stage 3 integration, use:

```
bryan/stage3-integration
```

**Suggested branch name**

```
bryan/stage3-integration
```

Copy

### Step 3: Create your branch

After typing the branch name, GitHub should display an option similar to:

**Create branch: bryan/stage3-integration from main**

Click that option.

Make sure the source branch is `main`, not `xinyi/stage-2-features`.

Your new branch will be created immediately.

### Step 4: Check that your branch was created

After creating it, look at the branch selector.

Instead of `main`, you should now see:

**bryan/stage3-integration**

Congratulations! Your branch is now separate from `main`.

You can make changes without directly affecting your teammates' work.

**Take your curiosity further**
Scan to get the ChatGPT app

### Step 5: Download your branch to your computer

If you've already cloned the repository, open Git Bash or the terminal in your project folder and enter:

```
git fetch origin
git switch bryan/stage3-integration
```

If Git says the branch doesn't exist locally, use:

```
git switch --track origin/bryan/stage3-integration
```

Then verify:

```
git branch
```

You should see something like:

```
* bryan/stage3-integration
  main
```

The `*` indicates your currently selected branch.

### Step 6: How to save your future changes

After editing your Java files:

```
git add .
git commit -m "Add Stage 3 integration changes"
git push -u origin bryan/stage3-integration
```

This uploads your changes to your own branch, not `main`.

When your feature is ready, you can create a Pull Request to merge it into `main`.

**Take your curiosity further**
Scan to get the ChatGPT app

### Which method do you want to use?

- ○ **GitHub website** — create a branch by clicking buttons (easiest)
- ○ **VS Code** — create and switch branches in your editor
- ○ **Git Bash / Terminal** — create a branch using commands

Guide me step by step

I recommend the **GitHub website** method if this is your first time creating a branch.

You

can i just use the codespace, how do i access to a codespace

ChatGPT

Yes! You can use **GitHub Codespaces** for your SC2002 group project. In fact, I recommend it because you can write, run and push your Java code directly from your browser without installing VS Code or Java on your computer.

I'll show you how to access it.

### Step 1: Open your GitHub repository

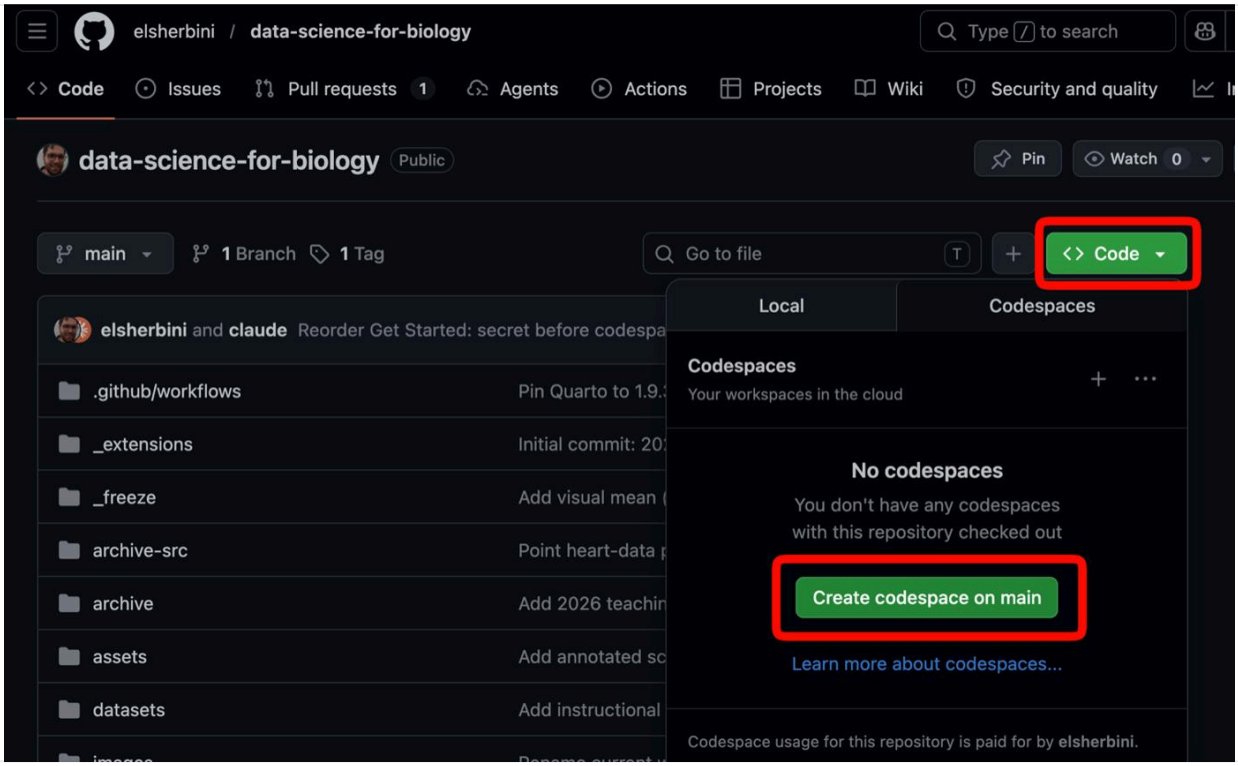
Go to your group's repository:

Open SC2002 Restaurant Rush repository ↗

Make sure you're signed in to your GitHub account.

**Take your curiosity further**
Scan to get the ChatGPT app

## Step 2: Click the green Code button




1. Click the green **Code** button.
2. Select the **Codespaces** tab.
3. Click **Create codespace on main**.

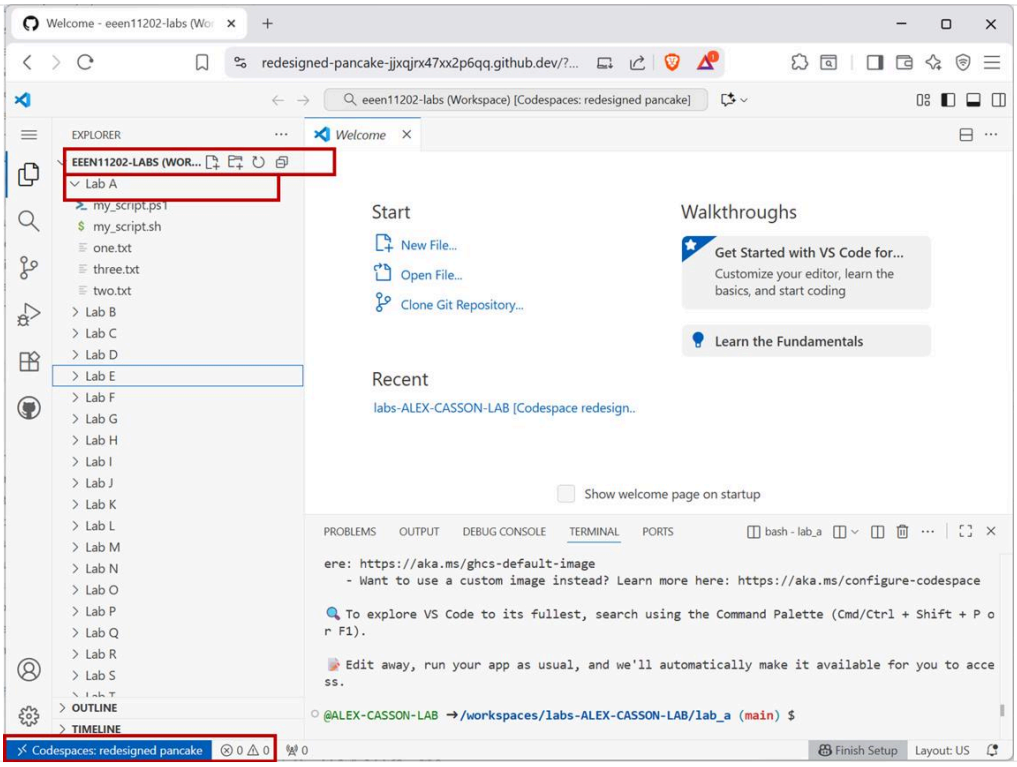
GitHub will start creating your development environment. This may take a minute or two.

If you already have a Codespace, you can select it from the list instead.

## Step 3: Wait for Codespaces to open

Your browser should open an editor that looks very similar to VS Code.

**Take your curiosity further**
Scan to get the ChatGPT app




You should see your project files on the left, including your Java source files.

## Step 4: Create your own branch inside Codespaces

Once Codespaces opens, click **Terminal → New Terminal**.

Enter these commands one at a time:

### 1. Switch to main

```
git switch main
```

### 2. Get the latest code from your teammates

```
git pull origin main
```

### 3. Create your own branch

For example:

**Take your curiosity further**
Scan to get the ChatGPT app

```
git switch -c bryan/stage3-integration
```

#### 4. Upload your branch to GitHub

```
git push -u origin bryan/stage3-integration
```

That's it! You've created your own branch and can work on it directly inside Codespaces.

### Step 5: Run your Java game

Your repository's README specifies Java 17 or newer.

In the Codespaces terminal, enter:

```
javac -d out src/*.java
```

Then:

```
java -cp out Game
```

If Java is available in the Codespace, your Restaurant Rush game should start running in the terminal.

### Step 6: How to reopen Codespaces next time

You don't need to create a new Codespace every time.

Go to:

Your GitHub Codespaces ↗

Find your SC2002 Codespace and click it to continue working.

**Important:** A Codespace is your development environment, while a Git branch tracks your code changes. They are different things. You can use the same Codespace while switching branches.

**Take your curiosity further**
Scan to get the ChatGPT app

**What I recommend you do now**

Open your group's GitHub repository, click **Code** → **Codespaces**, and check whether you see **Create codespace on main** or an existing Codespace.

You can send me a screenshot of what you see. I'll guide you through the next steps so you don't accidentally modify your teammates' `main` branch.

You

what are the sequence on workking on github, isit i branch out then edit then i pull request and what else, can you create a flow so that i know what to do next

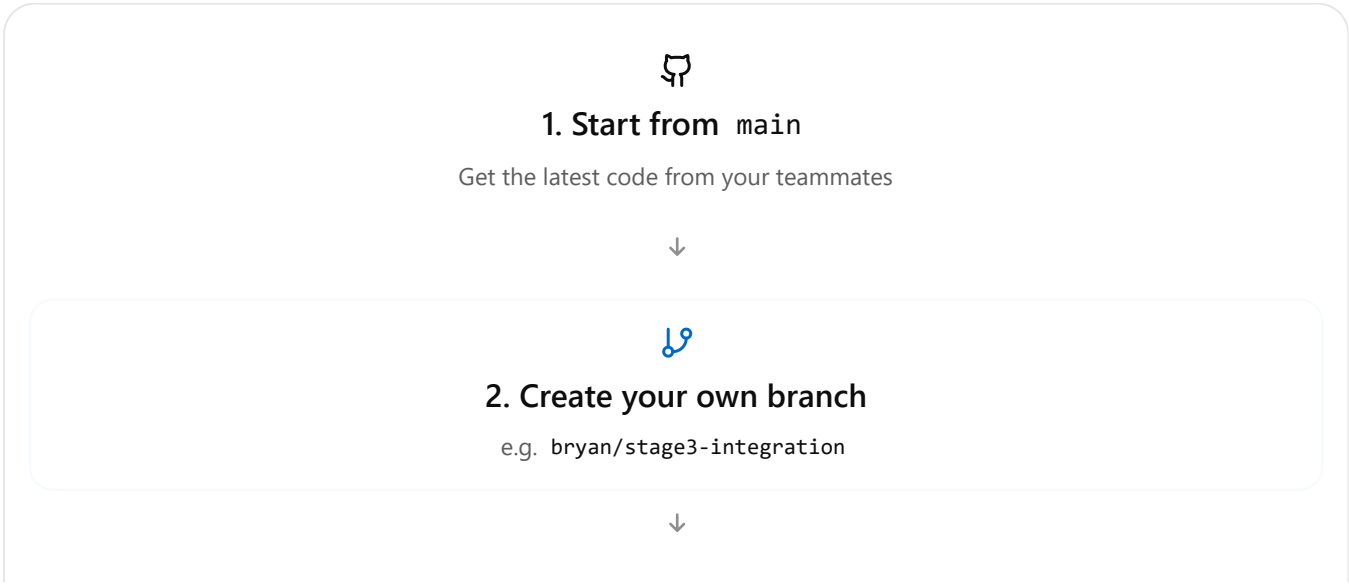
ChatGPT

Yes! For your **SC2002 Restaurant Rush group project**, the usual GitHub workflow is:

**Update `main` → Create branch → Edit code → Test → Commit → Push → Pull Request → Review → Merge → Repeat.**

Since you're using **GitHub Codespaces**, I'll explain exactly what to do at each stage.

## 1. Your complete GitHub workflow




Take your curiosity further
Scan to get the ChatGPT app

### 3. Edit your Java code

Make changes in Codespaces

### 4. Compile and test

Check that your changes work

### 5. Commit and push

Save changes and upload to GitHub

### 6. Open a Pull Request (PR)

Request to merge your work into `main`

### 7. Teammate reviews and tests

Fix issues if necessary, then get approval

### 8. Merge into `main`

Your code becomes part of the group's project

### 9. Update `main` and repeat

**Take your curiosity further**

Scan to get the ChatGPT app

Start a fresh branch for your next task

## 2. What you actually type in Codespaces

Here's an example using your own branch.

| Step | Command | Purpose |
|---|---|---|
| 1 | `git switch main` | Switch to main |
| 2 | `git pull origin main` | Get latest code |
| 3 | `git switch -c bryan/stage3-integration` | Create your branch |
| 4 | Edit Java files | Do your work |
| 5 | `javac -d out src/*.java` | Compile the game |
| 6 | `java -cp out Game` | Test the game |
| 7 | `git add .` | Stage your changes |
| 8 | `git commit -m "Integrate Stage 3"` | Save a commit |
| 9 | `git push -u origin bryan/stage3-integration` | Upload branch |

After Step 9, go to GitHub and open a Pull Request.

**You don't need to create a Pull Request after every small edit.** You can make multiple commits and pushes to your branch before opening a PR.

## 3. What happens after you open a Pull Request?

For example, suppose you finish integrating the Host feature.

Base branch
**main**

←

Compare branch
**bryan/stage3-integration**

**Pull Request: Integrate Stage 3 features**

Your teammate checks the changes, runs the game and reviews the code.

**Take your curiosity further**
Scan to get the ChatGPT app

- Approved and tests pass → Merge
- Changes requested → Fix, commit and push again

If a teammate asks you to fix something, **you do not need to create another PR**. Just edit your existing branch, commit and push. Your Pull Request updates automatically.

Once approved, click **Merge pull request**, then **Confirm merge**.

## 4. What should you do after merging?

This is the part beginners often forget!

After merging, go back to Codespaces and run:

```
git switch main
git pull origin main
```

Now your local `main` contains your merged changes and any new changes from your teammates.

For the next feature, create a new branch:

```
git switch -c bryan/next-feature
```

Avoid continuing new work on an old merged feature branch.

## 5. Your SC2002-specific workflow

Your assignment requires independently runnable Stage 1, Stage 2 and Stage 3 versions.

That means your team should also preserve the finished stage versions using Git tags:

- `stage-1` Basic restaurant game
- `stage-2` VIP and Critic added

**Take your curiosity further**
Scan to get the ChatGPT app

`stage-3` Host, ComboMeal, Happy Hour

A tag is different from a branch. A **branch moves forward** as you add commits, while a **tag identifies a specific saved version** of your code.

For example, your team should tag the verified Stage 2 commit, not the latest `main` commit if it already includes Stage 3 features.

### 6. Your personal checklist for every new feature

You can use this checklist every time you work on the group project.

**My GitHub workflow** 0/11 completed

- [ ] Switch to main
- [ ] Pull the latest main
- [ ] Create a new feature branch
- [ ] Edit my Java code
- [ ] Compile and test the game
- [ ] Check git status and commit
- [ ] Push my branch to GitHub
- [ ] Create a Pull Request
- [ ] Get teammate review and resolve issues
- [ ] Merge PR into main
- [ ] Switch back to main and pull again

Reset for next feature

**Take your curiosity further**
Scan to get the ChatGPT app

**One final tip:** For your five-person group, only merge a feature after checking that it doesn't break earlier stages. Your SC2002 grading also considers how your object-oriented design evolves between stages, so preserve your PR history, test evidence and UML diagrams.

The simplest rule to remember is:

**Pull before branching. Test before pushing. Review before merging. Pull again after merging.**

You

so right now im doin the part combomeal now, can you advice me on what to take note

Worked for 14s

ChatGPT

Since you're now responsible for **ComboMeal in your SC2002 Restaurant Rush group project**, I recommend focusing on three things:

1. Making sure ComboMeal satisfies the assignment requirements.
2. Designing it properly using Java OOP concepts.
3. Integrating it into your teammates' code without breaking Stage 1 or Stage 2.

The important thing is that **ComboMeal should work with your existing** `MenuItem`, `Order`, `Chef` **and** `Customer` **classes**, rather than requiring you to rewrite the whole game.

## 1. What exactly must ComboMeal do?

According to Section 4.3 of your Restaurant Rush assignment brief:

| Requirement | What your code must do |
|---|---|
| Composition | A ComboMeal contains two existing menu items |
| Pricing | 90% of the combined item prices |
| Preparation | Sum of both items' preparation units |
| Customer choice | Customers choose combos according to a documented rule |

**Take your curiosity further**
Scan to get the ChatGPT app

| | |
|---|---|
| Integration | ComboMeals follow the normal order, preparation, serving and payment flow |

For example:

### Example: Salad + Pasta Combo



**Salad**
$9.00 · 1 prep unit



**Pasta**
$18.00 · 2 prep units

| | |
|---|---|
| Original total | **$27.00** |
| Combo discount (10%) | **−$2.70** |
| **Combo price** | **$24.30** |

Preparation time: 1 + 2 = 3 units

This is the exact pricing example given in your assignment.

## 2. The OOP design you should use

I recommend making `ComboMeal` a subclass of `MenuItem`.

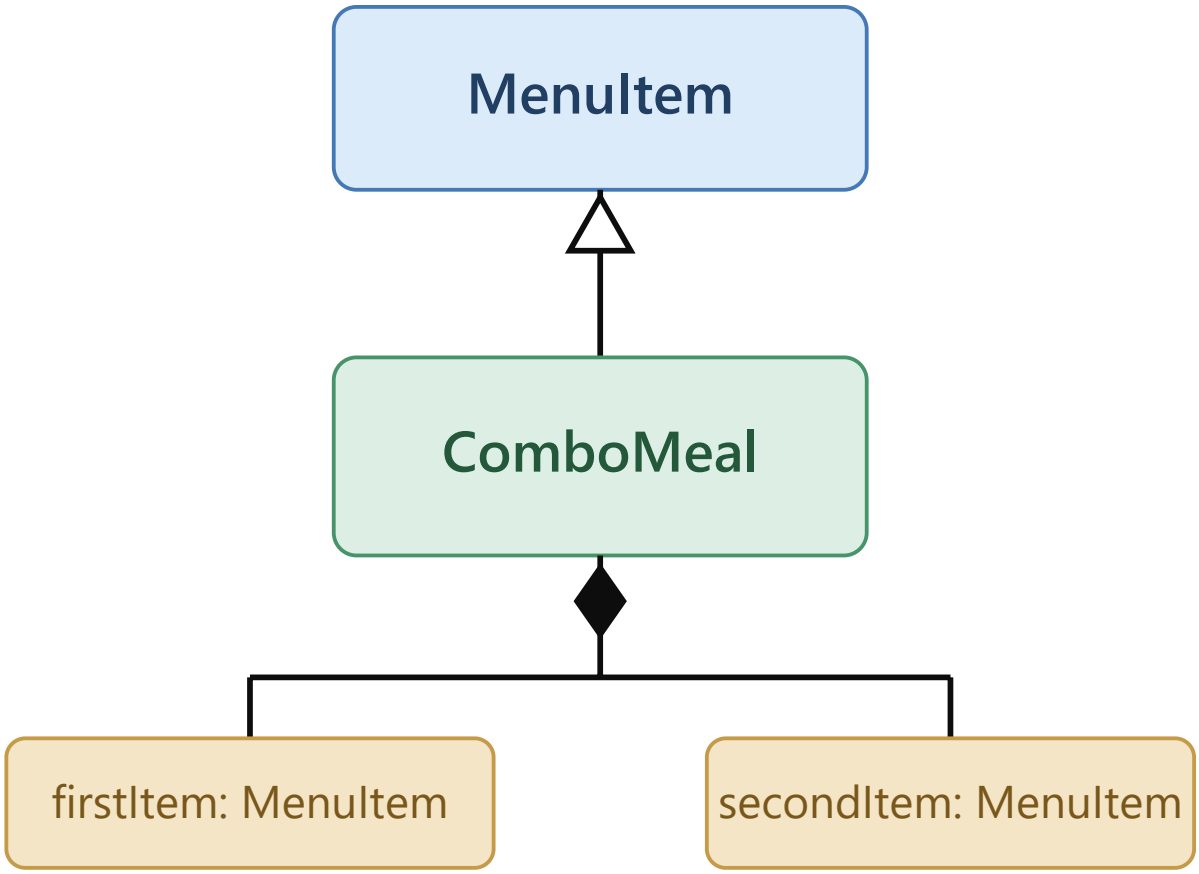
```
public class ComboMeal extends MenuItem {
    private final MenuItem firstItem;
```

**Take your curiosity further**
Scan to get the ChatGPT app

```
    private final MenuItem secondItem;
}
```

This demonstrates two important SC2002 concepts:

- **Inheritance:** `ComboMeal` is a type of `MenuItem`.
- **Composition:** `ComboMeal` contains two `MenuItem` objects.




Conceptual class structure. In your final UML, use proper inheritance and composition notation and show attributes and methods.

This approach is useful because `Order` already accepts a `MenuItem`. A `ComboMeal` can therefore be passed into `Order` without creating a separate order system.

### 3. Important: Your teammates have already implemented some ComboMeal code

I checked the current `main` branch on GitHub, and found:

**Take your curiosity further**
Scan to get the ChatGPT app

- `ComboMeal.java` already exists.
- `Menu.java` already has `addComboMeal()`.
- `GameController.java` already adds a Salad + Pasta combo.
- `Customer.java` uses `chooseMenuItem()` to select dishes.

This means **you shouldn't create another ComboMeal class from scratch**. Your task should be to review, test and improve the existing implementation.

Here are the main things I noticed.

### Issue 1 — ComboMeal input validation

The existing constructor assumes both menu items are valid.

For example, what happens if someone passes `null`? It could cause a `NullPointerException`.

You can add validation, but prioritise the required game behaviour first.

### Issue 2 — CriticCustomer may choose ComboMeal

Your Stage 2 Critic always chooses the most expensive dish.

Now that ComboMeal costs $24.30, it is more expensive than Pasta ($18.00).

Therefore, your Critic may start choosing the combo automatically.

This is consistent with the current most-expensive-item rule, but you should document it and test that Stage 2 behaviour remains correct in Stage 3.

### Issue 3 — Test your discount calculations

Here are some expected values:

| Scenario | Expected bill |
|---|---|
| Salad + Pasta Combo | $24.30 |
| VIP orders ComboMeal | $21.87 |

**Take your curiosity further**
Scan to get the ChatGPT app

| | |
|---|---|
| Regular orders ComboMeal during Happy Hour | $19.44 |
| VIP orders ComboMeal during Happy Hour | $17.50 |

For the final scenario:

$$2430 \times 0.9 = 2187 \text{ cents}$$

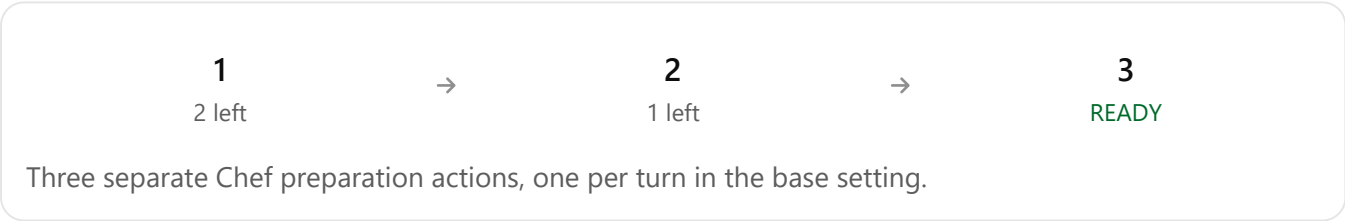
$$\text{round}(2187 \times 0.8) = 1750 \text{ cents}$$

The final bill is $17.50.

**Issue 4 — ComboMeal must take three preparation units**

A combo must not become READY after only one cooking action.

For Salad + Pasta:




Once READY, the Waiter can serve it on a subsequent turn, following the normal game rules.

## 4. What I recommend you do in Codespaces

First, update your local `main` and create a fresh branch:

```
git switch main
git pull origin main
git switch -c bryan/combomeal-review
```

Then examine these files:

**Take your curiosity further**
Scan to get the ChatGPT app

```
src/ComboMeal.java
src/Menu.java
src/MenuItem.java
src/Customer.java
src/CriticCustomer.java
src/GameController.java
```

Compile your project:

```
javac -d out src/*.java
```

Run the game:

```
java -cp out Game
```

Test whether a customer can order the combo, whether the Chef prepares it for three units and whether the Cashier collects the correct price.

Once you've made and tested improvements, commit and push them to your branch, then open a Pull Request.

## 5. Your ComboMeal checklist

**ComboMeal development checklist** 0/10

- [ ] ComboMeal extends MenuItem
- [ ] ComboMeal stores two component MenuItems
- [ ] Price is calculated as 90% of combined prices
- [ ] Preparation units are added correctly
- [ ] ComboMeal is registered in Menu
- [ ] Customers can select the combo
- [ ] Critic's most-expensive-item behaviour is checked

**Take your curiosity further**
Scan to get the ChatGPT app

- VIP and Happy Hour discounts work in the correct order
- ComboMeal follows normal cooking, serving and payment
- Tests and UML documentation are updated

**My suggestion:** Start by reviewing the existing ComboMeal implementation, rather than writing new code immediately. The current `main` already contains most of the basic functionality, so your contribution can focus on correctness, edge cases and integration testing.

The first thing I'd like you to do is open `ComboMeal.java` in Codespaces and confirm that it matches the version above. Then we can go through the class line by line and identify exactly what you should improve before creating your Pull Request.

You

this is what i dofor combomeal, the code compiles, but not sure whether the pricing, preparation units and object references are correct. can you help me review
if yes can you let me know what to fix first, dont generate the code for me first

```
public class ComboMeal extends MenuItem {

    private final MenuItem firstItem;
    private final MenuItem secondItem;

    public ComboMeal(MenuItem firstItem, MenuItem secondItem) {
        super(
            firstItem.getItemName() + " + " +
secondItem.getItemName(),
            calculatePrice(firstItem, secondItem),
            firstItem.getPreparationUnits()
        );

        this.firstItem = firstItem;
        this.secondItem = firstItem;
    }
```

**Take your curiosity further**
Scan to get the ChatGPT app

```
    private static int calculatePrice(MenuItem first, MenuItem second)
{
        int totalPrice = first.getPriceCents() + second.getPriceCents();
        return (int) Math.round(totalPrice * 0.8);
    }

    public MenuItem getFirstItem() {
        return firstItem;
    }

    public MenuItem getSecondItem() {
        return firstItem;
    }
}
```

ChatGPT

Yes! I've reviewed your `ComboMeal.java` against the **SC2002 Restaurant Rush Stage 3 requirements**.

Your code compiles, but there are **4 mistakes** affecting the pricing, preparation units and object references.

The good news is that your overall OOP structure is correct. You are using inheritance (`extends MenuItem`) and storing two `MenuItem` references.

## 1. Summary of mistakes

| Component | Your implementation | Status |
|---|---|---|
| Inheritance | `extends MenuItem` | ✓ Correct |
| First item reference | `this.firstItem = firstItem` | ✓ Correct |
| Second item reference | `this.secondItem = firstItem` | ✗ Wrong |
| Combo pricing | `totalPrice * 0.8` | ✗ Wrong |
| Preparation units | Only `firstItem.getPreparationUnits()` | ✗ Wrong |

**Take your curiosity further**
Scan to get the ChatGPT app

| | | |
|---|---|---|
| Second item getter | `return firstItem` | ⊗ Wrong |

## 2. What you should fix first (in order)

### Priority 1 — Fix the second object reference

Your constructor currently has:

```
this.secondItem = firstItem;
```

This means both fields refer to the same object.

For example, if you create a combo with Salad and Pasta, your object ends up storing Salad twice.

**What to change:** Make `secondItem` refer to the second constructor parameter, not the first.

### Priority 2 — Correct the pricing formula

Your code currently uses:

```
return (int) Math.round(totalPrice * 0.8);
```

Multiplying by `0.8` gives a 20% discount.

However, Stage 3 specifies a **10% discount**, meaning the customer pays 90% of the combined price.

For Salad ($9) + Pasta ($18):

$$(900 + 1800) \times 0.9 = 2430$$

Expected price: **$24.30**, not $21.60.

**What to change:** Correct the multiplier while keeping the rounded integer-cent result.

### Priority 3 — Fix the preparation units

Your constructor currently passes:

**Take your curiosity further**
Scan to get the ChatGPT app

`firstItem.getPreparationUnits()`

This only counts the first dish.

The assignment requires the preparation units of **both dishes to be added together**.

For Salad + Pasta:

$$1 + 2 = 3 \text{ preparation units}$$

**What to change:** Update the third argument of `super(...)` to account for both items.

**Priority 4 — Fix** `getSecondItem()`

Your method currently returns:

```
public MenuItem getSecondItem() {
    return firstItem;
}
```

This means calling `getSecondItem()` always gives you the first dish.

**What to change:** Return the field corresponding to the second dish.

## 3. What should happen after fixing everything?

Let's use Salad + Pasta to verify your implementation.

### ComboMeal test calculator

Select two dishes to see the expected results after your fixes.

First item: Salad

Second item: Pasta

EXPECTED OUTPUT

**Salad + Pasta**

Take your curiosity further
Scan to get the ChatGPT app

Combo price

**$24.30**

Preparation units

**3**

`getFirstItem()` → Salad

`getSecondItem()` → Pasta

For the game, use two existing menu items according to your group's chosen combo rule.

## 4. Why does your code compile even though it's wrong?

This is an important SC2002 concept.

Java checks whether your code is **syntactically and type correct**, but it does not automatically know your assignment's business rules.

For example:

```
this.secondItem = firstItem;
```

Both variables have type `MenuItem`, so Java considers this valid.

However, logically, you assigned the wrong object.

This is called a **logic error**, not a compilation error.

The same applies to your pricing multiplier and preparation units.

## 5. What to do after fixing the four mistakes

Once you've corrected them, I recommend testing these four conditions:

- `getFirstItem()` returns the original first `MenuItem` object.
- `getSecondItem()` returns the original second `MenuItem` object.
- `getPriceCents()` returns the discounted price in cents.
- `getPreparationUnits()` returns the sum of both dishes' preparation units.

**Take your curiosity further**
Scan to get the ChatGPT app

For the object references, check that they refer to the **same original objects** passed into the constructor, not merely objects with matching names.

One additional improvement for later is validating `null` inputs, but that is not as urgent as the four mistakes above.

**Your next step:** Fix Priority 1 first ( `secondItem` assignment), then the pricing, preparation units and getter. Once you've updated your code, send me your revised `ComboMeal.java` , and I'll review it again without generating the answer for you.

You

```
public class ComboMeal extends MenuItem {

    private final MenuItem firstItem;
    private final MenuItem secondItem;

    public ComboMeal(MenuItem firstItem, MenuItem secondItem) {
        super(
            firstItem.getItemName() + " + " +
secondItem.getItemName(),
            calculatePrice(firstItem, secondItem),
            firstItem.getPreparationUnits()
        );

        this.firstItem = firstItem;
        this.secondItem = secondItem; // FIX 1
    }

    private static int calculatePrice(MenuItem first, MenuItem second)
{
        int totalPrice = first.getPriceCents() + second.getPriceCents();
        return (int) Math.round(totalPrice * 0.8);
    }

    public MenuItem getFirstItem() {
        return firstItem;
    }
```

**Take your curiosity further**
Scan to get the ChatGPT app

```
    public MenuItem getSecondItem() {
        return firstItem;
    }
}
```

ChatGPT

Good! You've successfully fixed **Fix 1 — the second object reference**. ✅

Let's review your updated `ComboMeal.java`.

## Current progress

| Fix | Issue | Status |
|---|---|---|
| 1 | `this.secondItem = secondItem;` | ✓ Fixed |
| 2 | Incorrect pricing multiplier | ✗ Not fixed |
| 3 | Incorrect preparation units | ✗ Not fixed |
| 4 | Incorrect `getSecondItem()` | ✗ Not fixed |

**Progress** 1/4 fixes

## Next: Fix 2 — ComboMeal pricing

Look at your current method:

```
private static int calculatePrice(MenuItem first, MenuItem second) {
    int totalPrice = first.getPriceCents() + second.getPriceCents();
    return (int) Math.round(totalPrice * 0.8);
}
```

Your addition is correct, but your multiplier is wrong.

**Take your curiosity further**
Scan to get the ChatGPT app

According to the SC2002 assignment, **ComboMeal should cost 90% of the combined price**, not 80%.

For example:

- Salad = 900 cents
- Pasta = 1800 cents
- Combined price = 2700 cents
- Required combo price = 2430 cents ($24.30)

Your current calculation produces 2160 cents ($21.60).

**Hint:** You only need to change one number in `calculatePrice()`.

Once you've fixed that, we can move on to Fix 3 (preparation units).

I also noticed that `getSecondItem()` still returns `firstItem`, but we can address that after the preparation units.

**Try fixing the pricing yourself and send me your updated code.** I'll check it before we proceed, without giving you the complete solution.

You

```
public class ComboMeal extends MenuItem {

    private final MenuItem firstItem;
    private final MenuItem secondItem;

    public ComboMeal(MenuItem firstItem, MenuItem secondItem) {
        super(
            firstItem.getItemName() + " + " +
secondItem.getItemName(),
            calculatePrice(firstItem, secondItem),
            firstItem.getPreparationUnits() +
secondItem.getPreparationUnits()
        );

        this.firstItem = firstItem;
        this.secondItem = secondItem;
```

**Take your curiosity further**
Scan to get the ChatGPT app

```
    }

    private static int calculatePrice(MenuItem first, MenuItem second)
{
        int totalPrice = first.getPriceCents() + second.getPriceCents();
        return (int) Math.round(totalPrice * 0.9);
    }

    public MenuItem getFirstItem() {
        return firstItem;
    }

    public MenuItem getSecondItem() {
        return secondItem;
    }
}
```

is this ok

ChatGPT

Yes! ✅ Your updated `ComboMeal.java` is now **correct for all four issues we identified**.

You've fixed the pricing, preparation units and object references according to your SC2002 Stage 3 requirements.

## 1. Final code review

| Component | Your implementation | Status |
|---|---|---|
| Inheritance | `extends MenuItem` | ✓ Correct |
| First item reference | `this.firstItem = firstItem` | ✓ Correct |
| Second item reference | `this.secondItem = secondItem` | ✓ Correct |
| Pricing | `Math.round(totalPrice * 0.9)` | ✓ Correct |
| Preparation units | Sum of both items | ✓ Correct |

**Take your curiosity further**
Scan to get the ChatGPT app

| | | |
|---|---|---|
| First item getter | `return firstItem` | ✓ Correct |
| Second item getter | `return secondItem` | ✓ Correct |

**Code review progress** 4/4 fixes completed

All previously identified logic errors are resolved.

## 2. Let's verify with an example

Suppose you create a ComboMeal containing Salad and Pasta.

| Property | Salad | Pasta | ComboMeal |
|---|---|---|---|
| Price | $9.00 | $18.00 | **$24.30** |
| Preparation units | 1 | 2 | **3** |
| Object reference | `firstItem` | `secondItem` | Both retained |

Your code correctly calculates:

$$(900 + 1800) \times 0.9 = 2430 \text{ cents}$$

And:

$$1 + 2 = 3 \text{ preparation units}$$

The two getters also return the original `MenuItem` references passed to the constructor.

## 3. What should you do next?

Your `ComboMeal` class looks correct, but **that doesn't necessarily mean the entire ComboMeal feature works in the game yet**.

I recommend these next steps:

**Take your curiosity further**
Scan to get the ChatGPT app

1. **Test ComboMeal independently.** Verify its name, price, preparation units and both object references.
2. **Check** `Menu.java`. Make sure ComboMeal can be added to the restaurant's menu.
3. **Check** `Customer.java`. Confirm that customers can actually select ComboMeal.
4. **Check** `Order.java`. Confirm that ComboMeal requires three Chef preparation actions.
5. **Test the full game.** Confirm that a ComboMeal can be prepared, served and paid for correctly.

One optional improvement is checking that neither constructor argument is `null`, because your current constructor assumes both items exist.

### My recommendation

**Don't open your Pull Request just yet.** Your class-level logic is correct, but we should test its integration first.

Since you're learning Java OOP, I suggest we review `Menu.java` next, especially how `addComboMeal()` works and how your ComboMeal becomes available for customers to order.

Send me your `Menu.java` code, and I'll review it the same way—**pointing out what is correct, what needs fixing and why, without generating replacement code unless you ask.**

You

**combomeal termi...**
Document

i have dont the menu and add combomeal, right now i test using the terminal, but i not sure whether if the output is correct, can yuo help me check

ChatGPT

Yes! I've gone through your **entire 18-turn terminal output** and compared it against the SC2002 Restaurant Rush Stage 3 requirements.

**Take your curiosity further**
Scan to get the ChatGPT app

**Your ComboMeal pricing, preparation units, ordering, cooking and payment all appear to be working correctly!** However, there are a few things you should check before opening your Pull Request.

## 1. Overall test results

### ComboMeal integration test passed

The terminal output supports the expected ComboMeal behaviour.

| Final revenue | Paid customers | Avg. satisfaction |
|---|---|---|
| $65.40 | 4 / 4 | 66.3 |

The game correctly reports VICTORY because all three objectives are met.

## 2. ComboMeal requirements — detailed review

| Requirement | Evidence from your output | Result |
|---|---|---|
| ComboMeal added to menu | Turn 1 intro displays Salad + Pasta | ✓ Pass |
| Correct price | $24.30 displayed | ✓ Pass |
| Correct preparation units | 3 prep units displayed | ✓ Pass |
| Customer can order combo | Critic orders combo on Turn 9 | ✓ Pass |
| Chef prepares combo correctly | 3 preparation actions | ✓ Pass |
| Waiter serves combo | Turn 13 | ✓ Pass |
| Cashier collects payment | Turn 15, $24.30 | ✓ Pass |
| Correct object references | Not directly shown in terminal | ⊖ Needs separate test |

### A. Pricing — correct

Your output shows:

```
Salad + Pasta  $24.30  3 prep units
```

**Take your curiosity further**
Scan to get the ChatGPT app

This is correct because:

$$(900 + 1800) \times 0.9 = 2430 \text{ cents}$$

On Turn 9, the Critic orders Salad + Pasta for $24.30, and on Turn 15 the Cashier collects exactly $24.30.

The game also correctly excludes the unpaid combo ordered on Turn 18 from final revenue.

**B. Preparation units — correct**

Your output shows the following progression:

1. **Turn 9 — First preparation**
   `3 units → 2 units remaining`
2. **Turn 10 — Second preparation**
   `2 units → 1 unit remaining`
3. **Turn 11 — Chef prepares Burger instead**
   Combo remains at 1 unit remaining. This is correct because the Chef can work on only one order per turn.
4. **Turn 12 — Final preparation**
   `1 unit → READY`
5. **Turn 13 — Waiter serves ComboMeal**
   Correct: it can only be served after becoming READY.
6. **Turn 15 — Cashier collects $24.30**
   Correct: the customer finishes eating before payment.

This is a particularly good test because it shows that **ComboMeal does not require three consecutive Chef turns**. Preparation progress is preserved when the Chef works on another order.

**Take your curiosity further**
Scan to get the ChatGPT app

### C. Critic ordering behaviour — correct

On Turn 9:

```
Customer 3 (Critic) at Table 1 orders
Salad + Pasta ($24.30, 3 prep units).
```

Your Critic chooses the most expensive item, and ComboMeal is now more expensive than the original dishes.

Therefore, this is consistent with your existing Stage 2 Critic behaviour.

## 3. Final game result — correct

Your paid customers are:

| Customer | Payment | Satisfaction |
|---|---|---|
| Regular 1 | $18.00 | 68 |
| VIP 2 | $8.10 | 85 |
| Critic 3 | $24.30 | 28 |
| Regular 4 | $15.00 | 84 |
| **Total** | **$65.40** | **66.25 average** |

The average satisfaction is:

$$\frac{68 + 85 + 28 + 84}{4} = 66.25$$

Displayed to one decimal place, this becomes 66.3.

Your game meets all three objectives:

- At least 4 paid customers: 4.
- Revenue at least $45: $65.40.
- Average satisfaction at least 60: 66.3.

**Take your curiosity further**
Scan to get the ChatGPT app

So the **VICTORY result is correct**.

## 4. What you still need to check

Your terminal run proves that the ComboMeal works in this scenario, but it doesn't cover every possible case.

I recommend these remaining tests, in priority order.

**Remaining test checklist** 0/7

- [ ] Verify getFirstItem() and getSecondItem() return the original objects
- [ ] Test a VIP ordering ComboMeal (expected $21.87)
- [ ] Test a ComboMeal during Happy Hour (expected $19.44 for a regular customer)
- [ ] Test a VIP ComboMeal during Happy Hour (expected $17.50)
- [ ] Test menu behaviour when the menu is already full
- [ ] Check the documented customer combo-selection rule
- [ ] Run Stage 1 and Stage 2 regression tests

One thing to note: **your terminal output does not show the Stage 3 Host or Happy Hour features.** That's fine if this run was intentionally testing ComboMeal in isolation, but the full Stage 3 game must eventually include them.

Also, the terminal output cannot verify the actual `firstItem` and `secondItem` references. Your previously shared corrected class looks right, but a small object-identity test would confirm it.

## 5. What should you fix first?

I don't see a ComboMeal pricing or preparation bug that needs fixing based on this output.

I would prioritise **testing the interaction between ComboMeal, VIP discounts and Happy Hour**, because the Stage 3 requirements specify the order in which discounts must be applied.

After that, verify the object references and run the earlier-stage regression tests.

**Take your curiosity further**
Scan to get the ChatGPT app

**My recommendation:** Don't change your `ComboMeal.java` unnecessarily. Your core implementation appears correct. The next useful step is testing the edge cases, then preparing your GitHub Pull Request with evidence of the successful run.

If you share your updated `Menu.java`, I can also review whether the combo is registered safely, whether duplicate combos are handled properly and whether customer selection works as intended—without generating code for you.

You

moving to the next step how can i pull request now

ChatGPT

Yes! Since you've finished your **ComboMeal feature** and tested it in the terminal, the next step is to **commit your changes, push your branch, and create a Pull Request (PR)**.

I'll guide you using **GitHub Codespaces**, since that's what you're using for your SC2002 project.

## Step 1: Check which branch you're on

Open your Codespaces terminal and type:

```
git branch --show-current
```

You should see your own feature branch, for example:

```
bryan/combomeal-review
```

> **Important**
>
> If it says `main`, don't commit yet! Your changes should be committed on your own feature branch, not directly to `main`.

**Take your curiosity further**
Scan to get the ChatGPT app

### Step 2: Check which files you've changed

Enter:

```
git status
```

You might see something like:

```
modified:   src/ComboMeal.java
modified:   src/Menu.java
```

Check that these are the files you intended to change.

If you see unrelated files, don't include them in your commit.

### Step 3: Add your changes

For your ComboMeal feature, you can add the relevant files individually:

```
git add src/ComboMeal.java src/Menu.java
```

Only include files you've actually changed. If you modified other relevant Java files or added tests, stage those separately too.

Then commit:

```
git commit -m "Implement and test ComboMeal feature"
```

This saves your changes locally in your feature branch.

### Step 4: Push your branch to GitHub

Enter:

```
git push -u origin HEAD
```

**Take your curiosity further**
Scan to get the ChatGPT app

This uploads your current branch and its commits to GitHub.

**After a successful push**

Your ComboMeal changes are now available on your GitHub branch, but they are **not yet merged into** `main`.

## Step 5: Create your Pull Request on GitHub

Go to your group's repository:

SC2002 Restaurant Rush repository ↗

**1. Open Pull Requests**

Click the **Pull requests** tab near the top of the repository.

**2. Click New pull request**

Alternatively, GitHub may show a **Compare & pull request** button after you push.

**3. Select your branches**

Set `base: main` and `compare: your-combomeal-branch`.

**4. Add your title and description**

Explain what you implemented and how you tested it.

## Step 6: What should you write in your PR?

You can copy this description and edit it to match the changes you actually made.

Writing

**Title: Implement and verify Stage 3 ComboMeal feature**

### Summary

Implemented and tested the ComboMeal feature for Stage 3 of Restaurant Rush.

### Changes

- Corrected ComboMeal pricing to 90% of the combined menu item prices.
- Ensured preparation units are the sum of both component items.
- Corrected references to the first and second menu items.
- Verified ComboMeal registration and ordering through the restaurant menu.

### Testing

- Java source files compiled successfully.
- Verified Salad + Pasta ComboMeal costs $24.30 and requires 3 preparation units.
- Verified the Chef completes the combo over three preparation actions.
- Verified the Waiter serves the combo after it becomes READY.
- Verified the Cashier collects $24.30.
- Completed an 18-turn game with a VICTORY result.

### Remaining Checks

- VIP and Happy Hour discount combinations.
- Regression testing with the completed Stage 3 features.

## Step 7: Review before creating the PR

Before clicking **Create pull request**, check the **Files changed** tab.

Make sure it contains only your intended changes.

This is especially important for your project because `main` already contains some ComboMeal implementation. If your PR shows unexpected changes to other teammates' work, review those differences before proceeding.

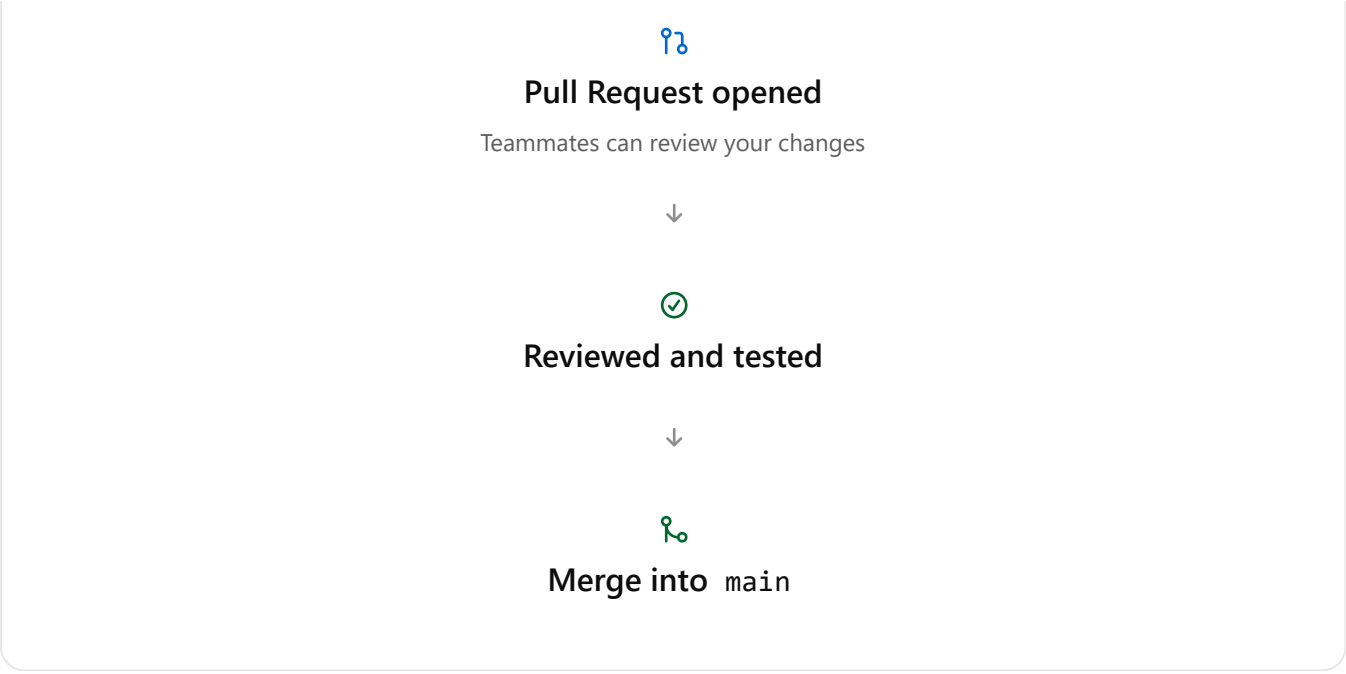
Then click **Create pull request**.

## Step 8: What happens next?

**Your ComboMeal branch**

↓

**Take your curiosity further**
Scan to get the ChatGPT app




Once your teammate approves the PR and the checks pass, it can be merged into `main`.

After merging, return to Codespaces and run:

```
git switch main
git pull origin main
```

Your local `main` will then contain the latest merged work.

### What I want you to do first

Before committing anything, open your Codespaces terminal and run these two commands:

```
git branch --show-current
git status
```

**Send me a screenshot or paste the output.** I'll check whether you're on the correct branch and which files you should commit, so you don't accidentally include your teammates' changes.

**Take your curiosity further**
Scan to get the ChatGPT app

You

i see a draft pull whats the difference

ChatGPT

On GitHub, there are **two types of Pull Requests (PRs)**: a normal Pull Request and a Draft Pull Request.

The main difference is **whether your code is ready for your teammates to review and merge**.

## 1. Normal PR vs Draft PR

| | Normal PR | Draft PR |
|---|---|---|
| Purpose | Ready for review | Still working on it |
| Teammates can see code | Yes | Yes |
| Teammates can comment | Yes | Yes |
| Can push more changes | Yes | Yes |
| Ready for formal review | Yes | Not yet |
| Can merge into `main` | Yes, if permitted | No, until marked ready |

## 2. Which should you choose for ComboMeal?

For your SC2002 project, here's what I recommend:

**Normal Pull Request — Recommended for you**

Choose this if you've finished your ComboMeal implementation and want your teammates to review it before merging.

Since you've already tested the pricing, preparation units, cooking and payment, this is a reasonable choice, provided your remaining checks are clearly documented.

**Draft Pull Request — If you're still developing**

**Take your curiosity further**
Scan to get the ChatGPT app

Choose this if you're still testing VIP discounts, Happy Hour integration or fixing ComboMeal bugs.

Your teammates can see your progress without treating the feature as ready to merge.

### 3. Can you change a Draft PR into a Normal PR later?

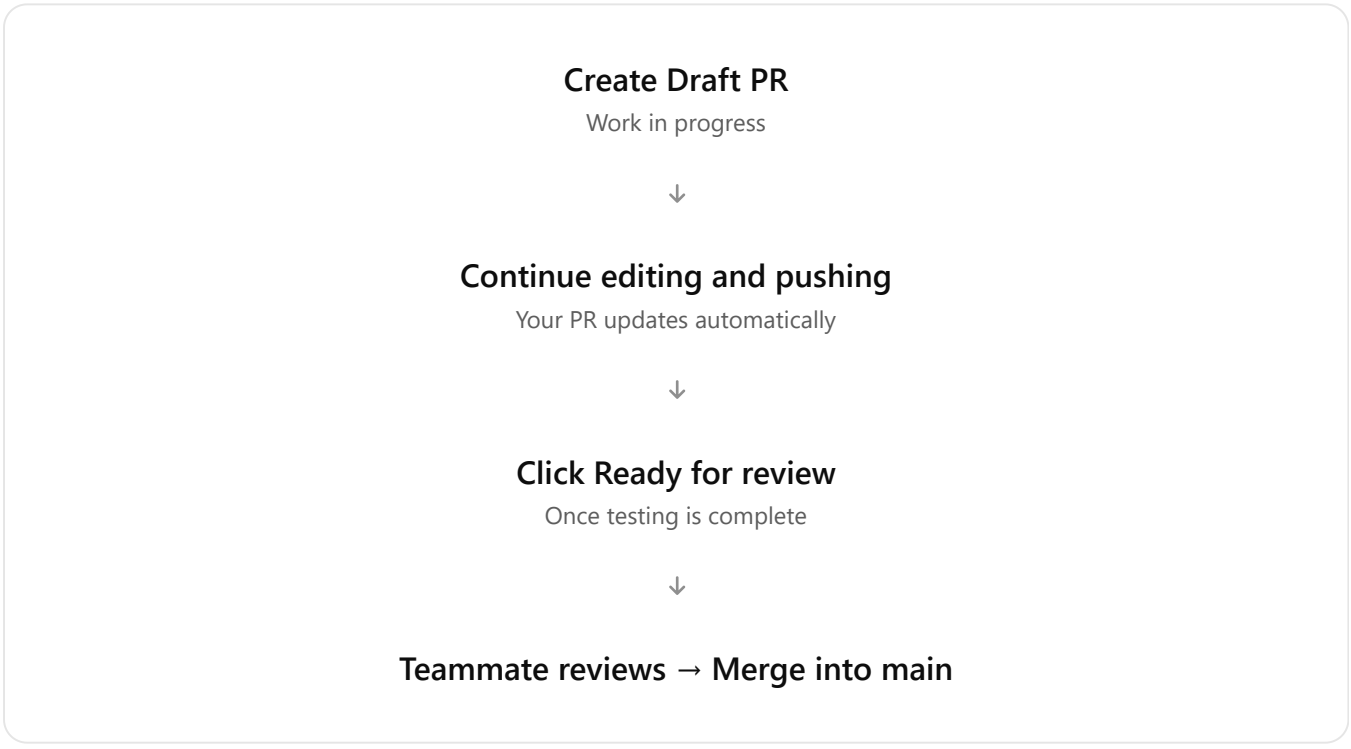
**Yes!** You don't need to create another Pull Request.

1. Open your Draft PR on GitHub.
2. Scroll down to the section near the bottom of the conversation.
3. Click **Ready for review**.
4. Your Draft PR becomes a normal PR.

You can also convert a normal PR into a draft if you discover more work is needed.

### 4. What happens if you select Draft PR?

Your workflow becomes:




Take your curiosity further
Scan to get the ChatGPT app

**My recommendation for your current situation**

Since your ComboMeal feature has passed its main gameplay test but you haven't verified every edge case yet, **I would choose Draft PR first**.

That way, your teammates can see your work while you finish testing. Once everything is ready, click **Ready for review**.

One important thing: **Creating either type of PR does not immediately change** `main`**.** Your code only enters `main` when the PR is actually merged.

You

mean while while waiting for my friend to complete the rest of stage 3, im doin the presentation video as welll, can you help me plan out a timeline what i should capture at what time, how the flow should go

Worked for 38s

ChatGPT

Yes! Since you're waiting for your teammates to finish the remaining Stage 3 features, this is a good time to prepare your **SC2002 Restaurant Rush presentation video**.

I've checked your *SC2002 Group Project Briefing*, especially page 11, which specifies a **15-minute recorded presentation**.

I recommend making the video a combination of **live gameplay recordings, UML diagrams, GitHub evidence and short explanations from all five members**.

The key is to avoid recording all 18 turns continuously. Instead, capture important moments and edit them into a clear demonstration.

## 1. Recommended 15-minute video timeline

TOTAL VIDEO DURATION

**15:00**

**Take your curiosity further**
Scan to get the ChatGPT app

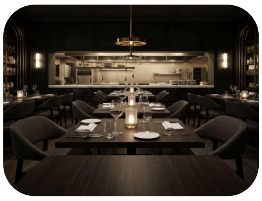
00:00 15:00

| Time | Section | Duration |
|---|---|---|
| 00:00–02:00 | Game overview and three stages | 2 min |
| 02:00–06:00 | Gameplay demonstration | 4 min |
| 06:00–10:00 | Class diagrams and design changes | 4 min |
| 10:00–13:00 | Sequence diagram and use case | 3 min |
| 13:00–15:00 | Individual contributions and learning | 2 min |

This follows the suggested allocation in your official project briefing.

## 2. Detailed storyboard — what to record at each timestamp

### Part 1: Game overview (00:00–02:00)



**00:00–00:30 — Introduction**

Show a title slide: *Restaurant Rush — SC2002 Object-Oriented Design & Programming*.

Include your five members' names and briefly explain that the player manages a restaurant with limited staff and turns.

18 turns

**00:30–01:10 — Game objectives**

Show the game's starting terminal screen.

Explain the 18-turn limit, two tables, Waiter, Chef, Cashier, and the victory requirements: four paid customers, $45 revenue and average satisfaction of at least 60.

3 stages

**01:10–02:00 — Development stages**

Show a simple three-stage timeline:

Stage 1: Basic restaurant operations.

Stage 2: VIP and Critic customers.

**Take your curiosity further**
Scan to get the ChatGPT app

Stage 3: Host, ComboMeal and Happy Hour.

Explain that each stage extends the previous version while preserving existing functionality.

## Part 2: Gameplay demonstration (02:00–06:00)

This should be the most engaging part of your video. Show the actual terminal, not just screenshots of the source code.

| Video time | What to capture | What to explain |
|---|---|---|
| 02:00–02:35 | Turn 1, seating a customer | Waiter has one action per turn |
| 02:35–03:10 | Taking an order and Chef preparing food | Manager prioritises orders |
| 03:10–03:45 | VIP and Critic behaviours | Polymorphism and different customer behaviours |
| 03:45–04:30 | ComboMeal ordering and cooking | Two menu items, 10% discount, combined prep units |
| 04:30–05:10 | Host and Happy Hour | Additional seating and temporary pricing policy |
| 05:10–05:35 | READY → SERVED → PAID | Order lifecycle and payment timing |
| 05:35–06:00 | Victory and defeat summaries | Game outcomes and reproducible testing |

### For your ComboMeal part, use the recording you've already tested

Your previous terminal run gives you excellent evidence.

### ComboMeal demonstration sequence

1. **Turn 9 — Critic orders ComboMeal**
   Show `Salad + Pasta ($24.30, 3 prep units)`.
2. **Turns 9, 10 and 12 — Chef preparation**

**Take your curiosity further**
Scan to get the ChatGPT app

Show the progression from 3 to 2 to 1 to READY, including the turn when the Chef prioritises another order.

3 **Turn 13 — Waiter serves ComboMeal**
Show the READY order being served.

4 **Turn 15 — Cashier collects payment**
Show the $24.30 payment and table becoming free.

This lets you demonstrate both the correct implementation and a meaningful manager decision.

I would allocate about **40–45 seconds** to this feature.

### Part 3: UML class diagrams and design changes (06:00–10:00)

This section is especially important because SC2002 assesses your **object-oriented design**, not just whether the game works.

| Video time | What to show | Main explanation |
|---|---|---|
| 06:00–06:45 | Stage 1 class diagram | Basic `Customer`, `Order`, `Menu`, `Table`, staff and controller relationships |
| 06:45–07:45 | Stage 2 class diagram | `VIPCustomer` and `CriticCustomer` inheritance and polymorphism |
| 07:45–08:45 | Stage 3 class diagram | New `Host`, `ComboMeal` and `HappyHour` classes |
| 08:45–09:30 | GitHub comparison | Show what changed between stages |
| 09:30–10:00 | Design justification | Explain how the design supports extensions |

For your ComboMeal section, show the relationship between `MenuItem` and `ComboMeal`.

Explain that `ComboMeal` inherits from `MenuItem`, while containing two existing `MenuItem` objects. This allows existing order and preparation logic to handle a combo without needing a completely separate system.

**Take your curiosity further**
Scan to get the ChatGPT app

For GitHub evidence, capture these comparisons:

Stage 1 → Stage 2 comparison ↗

Stage 2 → Stage 3 comparison ↗

Check that the tags point to the correct runnable versions before recording.

### Part 4: Sequence diagram (10:00–13:00)

Your assignment requires one complex use case from Stage 2 or Stage 3.

I recommend using a **Stage 3 order-processing scenario involving ComboMeal and Happy Hour**, provided your group's final code supports the exact interactions you draw.

**Suggested sequence diagram scenario**

Manager requests order

↓

Customer selects ComboMeal

↓

Calculate VIP and Happy Hour discounts

↓

Order created and Chef prepares meal

↓

Waiter serves and Cashier processes payment

Conceptual scenario only. The submitted UML must reflect the actual Java methods, guards and message order.

Divide the three minutes into explaining the starting conditions, walking through the object interactions, and highlighting the design decisions such as polymorphism and discount ordering.

**Take your curiosity further**
Scan to get the ChatGPT app

### Part 5: Individual contributions (13:00–15:00)

All five members should explain part of the work, as required by the briefing.

One possible allocation is:

| Time | Presenter | Focus |
|---|---|---|
| 13:00–13:24 | Member 1 | Stage 1 implementation |
| 13:24–13:48 | Member 2 | Stage 2 customer behaviour |
| 13:48–14:12 | You | ComboMeal implementation |
| 14:12–14:36 | Member 4 | Host / Happy Hour |
| 14:36–15:00 | Member 5 | Integration, testing and lessons |

Adjust this to your group's actual contributions. Every member should briefly cover what they implemented, one challenge and what they learned.

## 3. What you can record now vs what must wait

Since your teammates haven't finished Stage 3, I recommend preparing the video in two phases.

**Record now**

- Title and game overview.
- Stage 1 and Stage 2 gameplay, using the correct tagged versions.
- Your ComboMeal demonstration from the tested run.
- Existing Stage 1 and Stage 2 class diagrams.
- Your individual contribution explanation.
- GitHub Stage 1 → Stage 2 comparison.

**Record after Stage 3 is integrated**

- Final Host gameplay.
- Happy Hour activation, recovery and discounts.

**Take your curiosity further**
Scan to get the ChatGPT app

- Combined Stage 3 victory and defeat runs.
- Final Stage 3 class diagram.
- Final sequence diagram.
- GitHub Stage 2 → Stage 3 comparison.

I suggest saving your clips with descriptive filenames such as `01-introduction.mp4`, `02-combomeal.mp4` and `03-stage2-uml.mp4`. That makes editing much easier.

## 4. How to record the terminal neatly

Since you're using Codespaces, you can record directly from your browser.

I recommend using Windows Snipping Tool's screen recording feature or OBS Studio.

For the terminal recording:

- Increase the terminal font size so the examiner can read the text.
- Maximise the terminal area and hide unnecessary panels.
- Record at 1080p if possible.
- Capture each important gameplay event separately.
- Edit out unnecessary waiting and repeated menu selections.
- Add short labels such as "Turn 9 — ComboMeal ordered".
- Avoid speeding up footage containing important text too much.

Your existing `combomeal terminal(1).txt` can help you identify the exact turns to recreate.

## 5. Final video preparation checklist

**Video production tracker** 0/15 completed

- [ ] Title slide and group introduction
- [ ] Game objectives and three stages
- [ ] Basic waiter and chef gameplay
- [ ] VIP and Critic demonstration

**Take your curiosity further**
Scan to get the ChatGPT app

- [ ] ComboMeal ordering and preparation
- [ ] Host demonstration
- [ ] Happy Hour demonstration
- [ ] Payment and final game results
- [ ] Stage 1 class diagram
- [ ] Stage 2 class diagram
- [ ] Stage 3 class diagram
- [ ] GitHub stage comparisons
- [ ] Sequence diagram explanation
- [ ] All five members' contributions
- [ ] Timestamps added to README

### 6. One important deadline reminder

Your project briefing specifies **Sunday, 15 November 2026, 11:59 PM Singapore time** as the submission deadline.

Since the video is 15 minutes long, I'd prioritise getting the gameplay recordings and each member's audio completed as soon as possible, leaving time for editing, checking that the code matches the diagrams, and uploading the final files.

**My recommendation:** You should act as the video editor and coordinator, but have each teammate record their own assigned explanation. That way, you can start assembling the video now without waiting for the entire Stage 3 implementation to finish.

The final video should feel like one continuous presentation, even though different members contributed recordings.

**Take your curiosity further**
Scan to get the ChatGPT app